

Code Explanation

Unit Tests for Utility Functions

The given code is a set of unit tests for utility functions used in data processing with Apache Spark. These tests cover the functionality of getting a Spark session, validating data, and applying Slowly Changing Dimension (SCD) Type 2 changes.

Overview

The code includes tests for three main utility functions: ``get_spark_session``, ``validate_data``, and ``apply_scd_type2_changes``. It utilizes Pytest fixtures to create Spark sessions and sample DataFrames for testing.

Step 1: Creating a Spark Session Fixture

This step involves creating a Pytest fixture to establish a Spark session for testing purposes. The fixture is defined with a session scope, meaning it will be created once per test session.

```
@pytest.fixture(scope="session")
def spark():
    return SparkSession.builder
        .appName("TestUtils")
        .master("local[1]")
        .getOrCreate()
```

Step 2: Defining Sample DataFrames

Two fixtures, ``sample_target_df`` and ``sample_source_df``, are created to generate sample DataFrames representing target and source data for SCD Type 2 operations.

```
@pytest.fixture
def sample_target_df(spark):
    data = [
        ("C001", "John Doe", "john@example.com", "North", "0001", "Laptop", 1200.0, 1,
         datetime.date(2023, 1, 15), True, datetime.datetime(2023, 1, 15, 10, 0), None),
        # ... other data ...
    ]
    columns = ["CustId", "Name", "EmailId", "Region", "OrderId", "ItemName",
               "PricePerUnit", "Qty", "Date", "IsActive", "StartDate", "EndDate"]
    return spark.createDataFrame(data, columns)
```

Step 3: Testing get_spark_session Function

The ``test_get_spark_session`` function tests whether the ``get_spark_session`` function returns a valid `SparkSession` instance.

```
def test_get_spark_session():
    spark = get_spark_session()
    assert spark is not None
```

```
assert isinstance(spark, SparkSession)
```

Step 4: Testing validate_data Function

The `test\_validate\_data` function checks the `validate\_data` function's behavior with both valid and invalid column names.

```
def test_validate_data(spark):
    data = [("1", "John"), ("2", "Jane")]
    df = spark.createDataFrame(data, ["id", "name"])
    assert validate_data(df, ["id", "name"]) == True
    with pytest.raises(ValueError):
        validate_data(df, ["id", "nonexistent"])
```

Step 5: Testing apply_scd_type2_changes Function

The `test\_apply\_scd\_type2\_changes` function tests the `apply\_scd\_type2\_changes` function by applying SCD Type 2 changes to the sample target DataFrame based on the sample source DataFrame.

```
def test_apply_scd_type2_changes(spark, sample_target_df, sample_source_df):
    result = apply_scd_type2_changes(
        sample_target_df,
        sample_source_df,
        join_columns=["CustId", "OrderId"],
        compare_columns=["Name", "EmailId", "Region"]
    )
    # ... assertions on result ...
    to_expire = result["to_expire"]
    assert to_expire.count() == 2
    # ... other assertions ...
```

Step 6: Verifying SCD Type 2 Change Results

The test function verifies that the `apply\_scd\_type2\_changes` function correctly identifies records to expire and insert.

```
expired_custids = [row["CustId"] for row in to_expire.select("CustId").collect()]
assert "C002" in expired_custids
assert "C003" in expired_custids
to_insert = result["to_insert"]
assert to_insert.count() == 3
insert_custids = [row["CustId"] for row in to_insert.select("CustId").collect()]
assert "C002" in insert_custids
assert "C003" in insert_custids
assert "C004" in insert_custids
```